K L Deemed to be University

DATA STRUCTURES AND ALGORITHMS - 2

CO1 Java Code - BST and AVL Trees

Course Outcome	CO1
Topic	Binary Search Trees and AVL Trees
Language	Java
Document Type	Separate Implementation Code PDF

Objective

- Implement plain BST insertion and search for hierarchical dictionary indexing.
- Implement AVL insertion and deletion with height maintenance and rotations.
- Show why AVL trees preserve O(log n) lookup after updates.

How to Run

Save the code in a file using the public class name shown in the program, compile it using javac, and run it using java. Example: javac FileName.java followed by java FileName.

Complete Java Code

```
import java.util.*;

/*
 * CO1 Topic: Binary Search Tree and AVL Tree
 * This program demonstrates:
 * 1. Plain BST insertion, search and inorder traversal.
 * 2. AVL insertion with automatic LL, RR, LR and RL rotations.
 * 3. AVL deletion with rebalancing after node removal.
 */
public class CO1_BST_AVL_Demo {

    static class Node {
        int key;
        int height;
        Node left;
        Node right;

        Node(int key) {
            this.key = key;
            this.height = 1;
        }
    }

    // ---------------- Plain BST section ----------------
    static Node insertBST(Node root, int key) {
        if (root == null) return new Node(key);
        if (key < root.key) root.left = insertBST(root.left, key);
        else if (key > root.key) root.right = insertBST(root.right, key);
        return root;
    }

    static boolean searchBST(Node root, int key) {
        while (root != null) {
            if (key == root.key) return true;
            root = (key < root.key) ? root.left : root.right;
        }
        return false;
    }

    // ---------------- AVL helper methods ----------------
    static int height(Node node) {
        return node == null ? 0 : node.height;
    }

    static int balanceFactor(Node node) {
        return node == null ? 0 : height(node.left) - height(node.right);
    }

    static void updateHeight(Node node) {
        if (node != null) {
            node.height = 1 + Math.max(height(node.left), height(node.right));
        }
    }

    static Node rotateRight(Node y) {
        Node x = y.left;
        Node t2 = x.right;

        x.right = y;
        y.left = t2;

        updateHeight(y);
        updateHeight(x);
        return x;
    }

    static Node rotateLeft(Node x) {
        Node y = x.right;
        Node t2 = y.left;

        y.left = x;
        x.right = t2;

        updateHeight(x);
        updateHeight(y);
        return y;
    }

    static Node insertAVL(Node root, int key) {
        if (root == null) return new Node(key);

        if (key < root.key) root.left = insertAVL(root.left, key);
        else if (key > root.key) root.right = insertAVL(root.right, key);
        else return root; // duplicates are ignored
```

```
        updateHeight(root);
        int bf = balanceFactor(root);

        // LL case
        if (bf > 1 && key < root.left.key) return rotateRight(root);

        // RR case
        if (bf < -1 && key > root.right.key) return rotateLeft(root);

        // LR case
        if (bf > 1 && key > root.left.key) {
            root.left = rotateLeft(root.left);
            return rotateRight(root);
        }

        // RL case
        if (bf < -1 && key < root.right.key) {
            root.right = rotateRight(root.right);
            return rotateLeft(root);
        }

        return root;
    }

    static Node minValueNode(Node root) {
        Node current = root;
        while (current.left != null) current = current.left;
        return current;
    }

    static Node deleteAVL(Node root, int key) {
        if (root == null) return null;

        if (key < root.key) root.left = deleteAVL(root.left, key);
        else if (key > root.key) root.right = deleteAVL(root.right, key);
        else {
            if (root.left == null || root.right == null) {
                root = (root.left != null) ? root.left : root.right;
            } else {
                Node successor = minValueNode(root.right);
                root.key = successor.key;
                root.right = deleteAVL(root.right, successor.key);
            }
        }

        if (root == null) return null;

        updateHeight(root);
        int bf = balanceFactor(root);

        if (bf > 1 && balanceFactor(root.left) >= 0) return rotateRight(root);
        if (bf > 1 && balanceFactor(root.left) < 0) {
            root.left = rotateLeft(root.left);
            return rotateRight(root);
        }
        if (bf < -1 && balanceFactor(root.right) <= 0) return rotateLeft(root);
        if (bf < -1 && balanceFactor(root.right) > 0) {
            root.right = rotateRight(root.right);
            return rotateLeft(root);
        }

        return root;
    }

    static void inorder(Node root) {
        if (root == null) return;
        inorder(root.left);
        System.out.print(root.key + " ");
        inorder(root.right);
    }

    static void levelOrder(Node root) {
        if (root == null) return;
        Queue<Node> q = new LinkedList<>();
        q.add(root);
        while (!q.isEmpty()) {
            int size = q.size();
            while (size-- > 0) {
                Node cur = q.poll();
                System.out.print(cur.key + "(BF=" + balanceFactor(cur) + ") ");
                if (cur.left != null) q.add(cur.left);
                if (cur.right != null) q.add(cur.right);
            }
            System.out.println();
        }
    }
```

```
    public static void main(String[] args) {
        int[] keys = {40, 20, 60, 10, 30, 50, 70, 25, 35, 5, 15, 27, 45};

        Node bstRoot = null;
        Node avlRoot = null;

        for (int key : keys) {
            bstRoot = insertBST(bstRoot, key);
            avlRoot = insertAVL(avlRoot, key);
        }

        System.out.print("BST inorder: ");
        inorder(bstRoot);
        System.out.println();

        System.out.print("AVL inorder: ");
        inorder(avlRoot);
        System.out.println();

        System.out.println("AVL level order with balance factors:");
        levelOrder(avlRoot);

        System.out.println("Search 27 in BST: " + searchBST(bstRoot, 27));
        System.out.println("Search 99 in BST: " + searchBST(bstRoot, 99));

        int[] deletions = {10, 60, 40};
        for (int key : deletions) {
            avlRoot = deleteAVL(avlRoot, key);
            System.out.println("After deleting " + key + ":");
            levelOrder(avlRoot);
        }
    }
}
```

Expected Sample Output

```
BST inorder: 5 10 15 20 25 27 30 35 40 45 50 60 70
AVL inorder: 5 10 15 20 25 27 30 35 40 45 50 60 70
AVL level order with balance factors:
30(BF=0)
20(BF=0) 50(BF=0)
10(BF=0) 25(BF=-1) 40(BF=0) 60(BF=-1)
5(BF=0) 15(BF=0) 27(BF=0) 35(BF=0) 45(BF=0) 70(BF=0)
Search 27 in BST: true
Search 99 in BST: false
After deleting 10:
... AVL remains balanced after height update and rotation if needed ...
After deleting 60:
... AVL remains balanced after height update and rotation if needed ...
After deleting 40:
... AVL remains balanced after height update and rotation if needed ...
```

Implementation Notes

- BST operations depend on tree height h; balanced trees keep h close to log n.
- AVL balance factor is height(left) - height(right), and valid values are -1, 0 and +1.
- Rotations are local pointer changes that restore balance without breaking sorted inorder order.